EDA

Ryann Tompkins, Roberta Southwood, Meg Norton, Maddy Stone

Loading Data

Summary stats and looking at target

Dataset: CPS Annual Social and Economic (March 2026) Supplement for DC

https://data.census.gov/app/mdat/CPSASEC2026/table?cv=NOW_COV,A_SEX,LJCW,PRWKSTAT&rv=ucgid,A_HGA,PRDISFLG&vv=A_AGE,HUNDER18,FTOTVAL&wt=MARSUPWT&g=AwFm-BVBlBGWg

Column codes: https://www2.census.gov/programs-surveys/cps/techdocs/cpsmar26.pdf

```
library(readr)
Insurance_data <- read_csv("InsuranceData_correctedcolumns.csv")
```

```
New names:
Rows: 1918 Columns: 11
-- Column specification
-------------------------------------------------------- Delimiter: "," dbl
(10): NOW_COV, A_SEX, LJCW, PRWKSTAT, A_HGA, PRDISFLG, A_AGE, HUNDER18, ... lgl
(1): ...11
i Use `spec()` to retrieve the full column specification for this data. i
Specify the column types or set `show_col_types = FALSE` to quiet this message.
* `` -> `...11`
```

```
View(Insurance_data)
```

Cleaning Data

```r
library(tidyverse)
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.1.4     v purrr     1.1.0
v forcats   1.0.0     v stringr   1.5.1
v ggplot2   4.0.0     v tibble    3.2.1
v lubridate 1.9.4     v tidyr     1.3.1
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becor
```

```r
df <- Insurance_data |>  select(where(~ !all(is.na(.x))))

# dropping state since all are from DC
df <- df|> select(-STATE)

# make rows into factors
df <- df |>
  mutate(
    NOW_COV  = factor(NOW_COV),
    A_SEX    = factor(A_SEX),
    LJCW     = factor(LJCW),
    PRWKSTAT = factor(PRWKSTAT),
    A_HGA    = factor(A_HGA),
    PRDISFLG    = factor(PRDISFLG)
  )

# rename columns for clarity, some are kept the same
df <- df |>
  rename(
    # NOW_COV: 1 = covered, 2 = not covered
    NOW_COV = NOW_COV,

    # sex: 1 = Male, 2 = Female
    SEX = A_SEX,

    # class of longest job
    # 0 = NIU, 1 = private, 2 = federal, 3 = state, 4 = local,
    # 5 = self employed incorporated, yes, 6 = self employed incorporated, no or farm
    # 7 = without pay
```

```
    LONGEST_JOB_CLASS = LJCW,

    # full or part time work
    # 00 = NIU, 01 = Not in labor force, 02 = FT hours (35+), usually FT
    # 03 = PT for economic reasons, usually FT, 04 = PT for non-economic reasons, usually FT
    # 05 = Not at work, usually FT, 06 = PT hrs, usually PT for economic reasons
    # 07 = PT hrs, usually PT for non-economic, 08 = FT hours, usually PT for economic reason
    # 09 = FT hours, usually PT for non-economic reasons, 10 = Not at work, usually part-time
    # 11 = Unemployed FT, 12 = Unemployed PT
    FULL_PART_TIME = PRWKSTAT,

    # highest grade achieved
    # 0 = Children, 31 = Less than 1st grade, 32 = 1st,2nd,3rd,or 4th grade
    # 33 = 5th or 6th grade, 34 = 7th and 8th grade, 35 = 9th grade
    # 36 = 10th grade, 37 = 11th grade, 38 = 12th grade no diploma
    # 39 = High school graduate - high school diploma or equivalent
    # 40 = Some college but no degree, 41 = Associate degree in college occupation program
    # 42 = Associate degree in college, 43 = Bachelor's degree, 44 = Master's degree,
    # 45 = Professional school degree, 46 = Doctorate degree
    EDUCATION = A_HGA,

    # disability
    # -1 = NIU, 1 = Yes, 2 = No
    DISABILITY = PRDISFLG,

    # age
    AGE = A_AGE,

    # Number of persons in household under age 18
    HUNDER18 = HUNDER18,

    # Total family income
    TOTAL_FAM_INCOME = FTOTVAL
  )


summary(df)
```

```
 NOW_COV  SEX      LONGEST_JOB_CLASS FULL_PART_TIME   EDUCATION   DISABILITY
 1:1856   1: 884   0:794             2      :802    43     :453   -1: 299
 2:  62   2:1034   1:790             1      :522    44     :400   1 : 210
                   2:158             0      :299    0      :288   2 :1409
```

```
                3: 29            4      : 87    39     :276
                4: 55            7      : 81    40     :125
                5: 54            11     : 50    45     :117
                6: 38            (Other): 77    (Other):259
      AGE           HUNDER18      TOTAL_FAM_INCOME
 Min.   : 0.0   Min.   :0.0000   Min.   :      0
 1st Qu.:25.0   1st Qu.:0.0000   1st Qu.:  47298
 Median :37.0   Median :0.0000   Median : 114587
 Mean   :39.7   Mean   :0.6694   Mean   : 180289
 3rd Qu.:55.0   3rd Qu.:1.0000   3rd Qu.: 245530
 Max.   :85.0   Max.   :5.0000   Max.   :1480199
```

Splitting Data

```r
# split into training and testing
set.seed(123)

# want to use 80-20 split
training_pct <- 0.8

# split the data
Z <- sample(nrow(df), floor(training_pct * nrow(df)))
df_training <- df[Z, ]
df_testing  <- df[-Z, ]

# view split
nrow(df_training)
```

```
[1] 1534
```

```r
nrow(df_testing)
```

```
[1] 384
```

EDA

```r
summary(df_training)
```

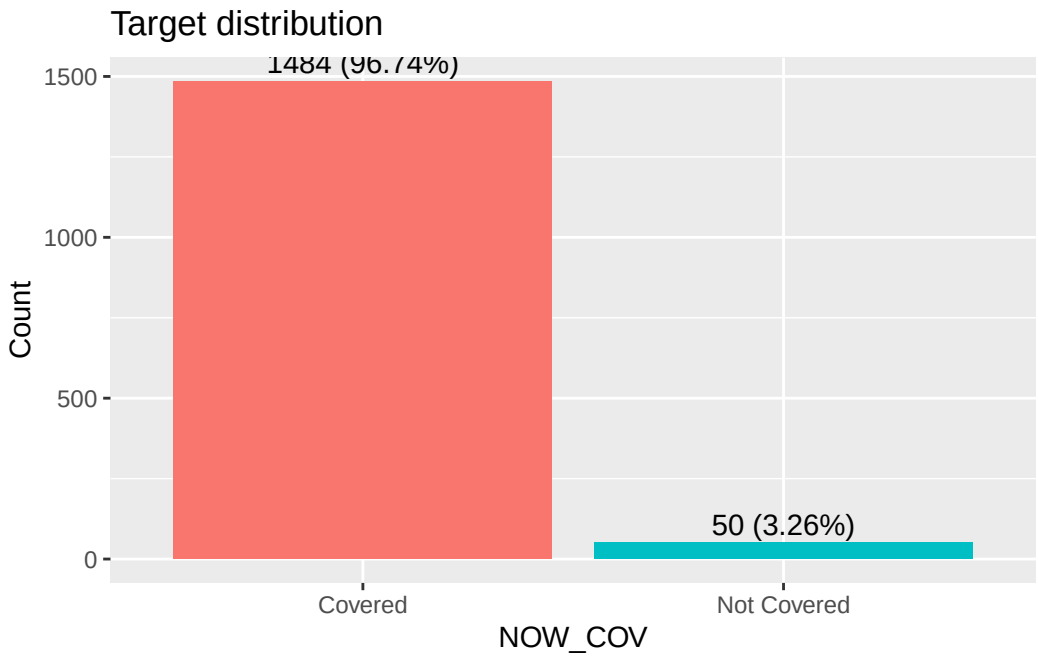
```
 NOW_COV  SEX     LONGEST_JOB_CLASS FULL_PART_TIME   EDUCATION   DISABILITY
 1:1484   1:702   0:637             2      :625    43     :355   -1: 252
 2:  50   2:832   1:627             1      :409    44     :317   1 : 157
                  2:123             0      :252    0      :242   2 :1125
                  3: 23             4      : 73    39     :224
                  4: 47             7      : 63    40     :110
                  5: 43             11     : 45    45     : 91
                  6: 34             (Other): 67    (Other):195
      AGE            HUNDER18      TOTAL_FAM_INCOME
 Min.   : 0.00   Min.   :0.0000   Min.   :       0
 1st Qu.:25.00   1st Qu.:0.0000   1st Qu.:  47000
 Median :37.00   Median :0.0000   Median : 111946
 Mean   :39.21   Mean   :0.6851   Mean   : 177522
 3rd Qu.:55.00   3rd Qu.:1.0000   3rd Qu.: 245316
 Max.   :85.00   Max.   :5.0000   Max.   :1480199
```

```r
target_tbl <- df_training |>
  count(NOW_COV) |>
  mutate(pct = round(100 * n / sum(n), 2))

print(target_tbl)
```

```
# A tibble: 2 x 3
  NOW_COV     n   pct
  <fct>   <int> <dbl>
1 1        1484 96.7
2 2          50  3.26
```

```r
ggplot(target_tbl, aes(factor(NOW_COV), n, fill = factor(NOW_COV))) +
  geom_col(show.legend = FALSE) +
  geom_text(aes(label = paste0(n, " (", pct, "%)")), vjust = -0.4) +
  scale_x_discrete(labels = c("1" = "Covered", "2" = "Not Covered")) +
  labs(title = "Target distribution", x = "NOW_COV", y = "Count")
```




```
#numeric pred. by summary group

df_training |>
  group_by(NOW_COV) |>
  summarise(
    n = n(),
    mean_age = mean(AGE), median_age = median(AGE),
    mean_income = mean(TOTAL_FAM_INCOME), median_income= median(TOTAL_FAM_INCOME),
    mean_under18 = mean(HUNDER18)
  )
```

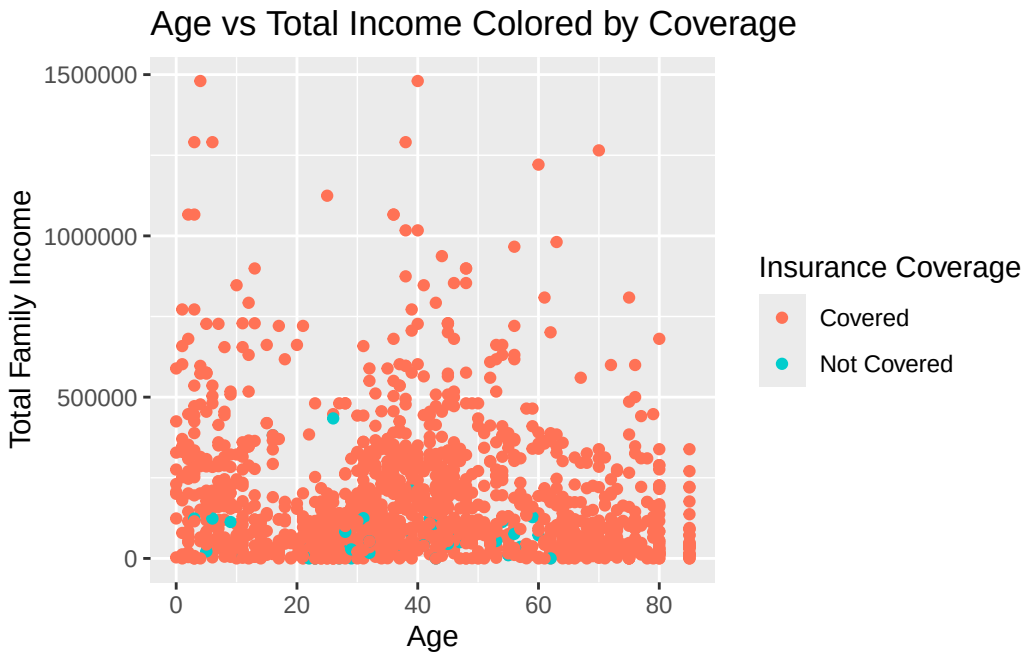
```
# A tibble: 2 x 7
  NOW_COV     n mean_age median_age mean_income median_income mean_under18
  <fct>   <int>    <dbl>      <dbl>       <dbl>         <dbl>        <dbl>
1 1        1484     39.4         37     181210.        115566        0.693
2 2          50     33.1         32      68064.         47691        0.44
```

```
# scatter plot of age vs income

ggplot(df_training, aes(x = AGE, y = TOTAL_FAM_INCOME, color = NOW_COV)) + geom_point() +
  labs(title = "Age vs Total Income Colored by Coverage",
       x = "Age", y = "Total Family Income") +
```

```
  scale_color_manual(name = "Insurance Coverage", values = c("1" = "coral1", "2" = "cyan3"),
  scale_size_manual(values = c("1" = 1.5, "2" = 6))
```

```
Warning: No shared levels found between `names(values)` of the manual scale and the
data's size values.
```



EDA: looking at which matter most

```
#install.packages("randomForest")
library(randomForest)
```

```
randomForest 4.7-1.2
```

```
Type rfNews() to see new features/changes/bug fixes.
```

```
Attaching package: 'randomForest'
```

```
The following object is masked from 'package:dplyr':

    combine

The following object is masked from 'package:ggplot2':

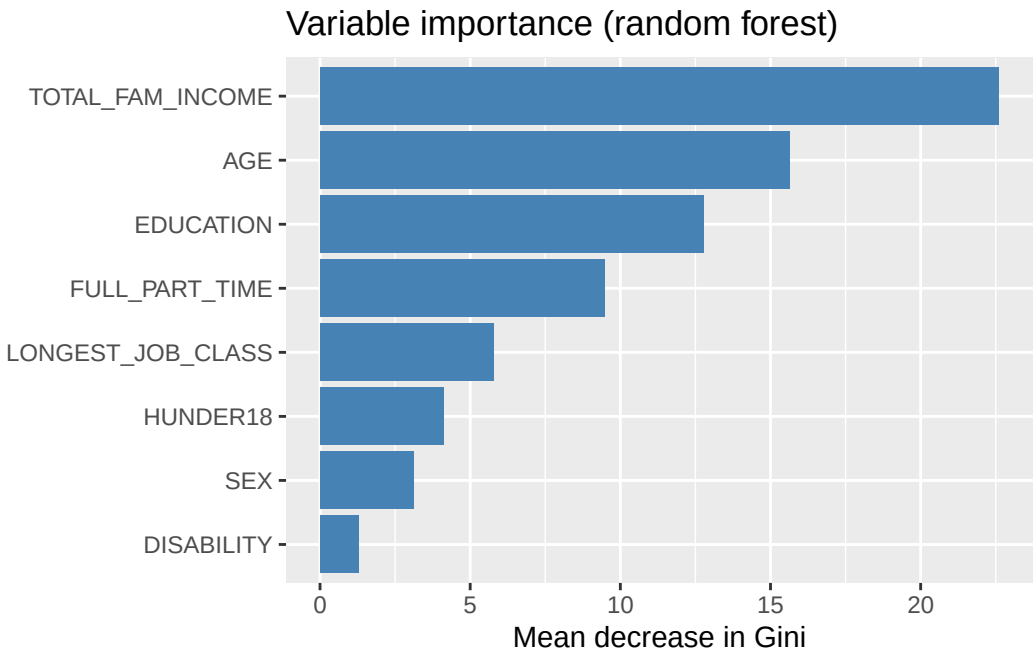
    margin
```

```r
rf <- randomForest(NOW_COV ~ ., data = df_training, ntree = 500, importance = TRUE)

imp <- importance(rf) %>%
  as.data.frame() %>%
  rownames_to_column("variable") %>%
  arrange(desc(MeanDecreaseGini))
print(imp)
```

```
           variable         1          2 MeanDecreaseAccuracy MeanDecreaseGini
1  TOTAL_FAM_INCOME 10.815744 10.5125106           12.9793512        22.607578
2               AGE 12.282570  6.3519728           13.2004985        15.628863
3         EDUCATION 22.680530  4.3281659           23.7686961        12.752755
4    FULL_PART_TIME 18.506054  0.2173085           18.8000089         9.459074
5 LONGEST_JOB_CLASS 13.644262  1.1543203           13.9182315         5.762413
6          HUNDER18  8.133738  5.7524163            8.8283947         4.117375
7               SEX -1.014017  4.0553291            0.3274109         3.129882
8        DISABILITY  6.033970 -1.3495343            6.0934674         1.265730
```

```r
ggplot(imp, aes(reorder(variable, MeanDecreaseGini), MeanDecreaseGini)) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  scale_x_discrete(labels = c("1" = "Covered", "2" = "Not Covered")) +
  labs(title = "Variable importance (random forest)", x = NULL,
       y = "Mean decrease in Gini")
```




```
top_vars <- head(imp$variable, 3)
print(top_vars)
```

```
[1] "TOTAL_FAM_INCOME" "AGE"              "EDUCATION"
```

```
for (v in top_vars) {
  cat("\n---", v, "---\n")

  if (is.numeric(df_training[[v]])) {
    # numeric: summary by target + boxplot
    print(df_training %>% group_by(NOW_COV) %>% summarise(
      min = min(.data[[v]]), median = median(.data[[v]]),
      mean = mean(.data[[v]]), max = max(.data[[v]])))

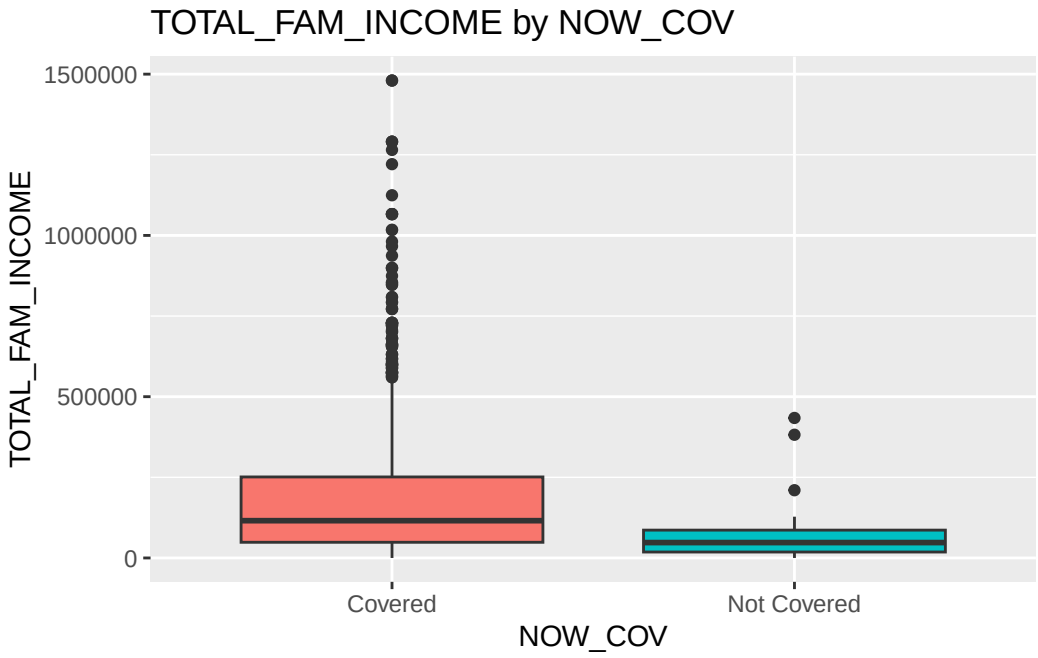
    p <- ggplot(df_training, aes(NOW_COV, .data[[v]], fill = NOW_COV)) +
      geom_boxplot(show.legend = FALSE) +
      scale_x_discrete(labels = c("1" = "Covered", "2" = "Not Covered")) +
      labs(title = paste(v, "by NOW_COV"), x = "NOW_COV")
    if (v == "FOTHVAL") p <- p + scale_y_continuous(trans = "pseudo_log")
    print(p)

  } else {
```

```
    # categorical: proportion table + stacked bar
    print(round(prop.table(table(df_training[[v]], df_training$NOW_COV), margin = 1), 3))

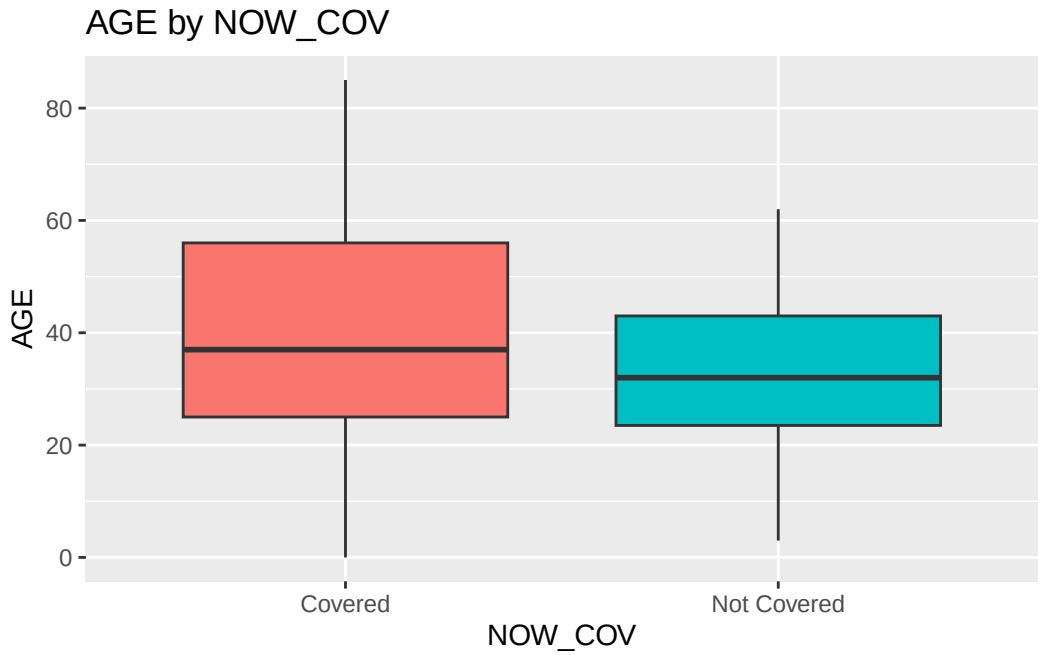
    print(ggplot(df_training, aes(.data[[v]], fill = NOW_COV)) +
            geom_bar(position = "fill") +
            labs(title = paste("NOW_COV proportion by", v),
                 subtitle = "1 = covered, 2 = not covered", x = v, y = "Proportion"))
    }
}
```

```
--- TOTAL_FAM_INCOME ---
# A tibble: 2 x 5
  NOW_COV   min median    mean     max
  <fct>   <dbl>  <dbl>   <dbl>   <dbl>
1 1           0 115566 181210. 1480199
2 2           0  47691  68064.  434100
```




```
--- AGE ---
# A tibble: 2 x 5
  NOW_COV   min median  mean   max
```

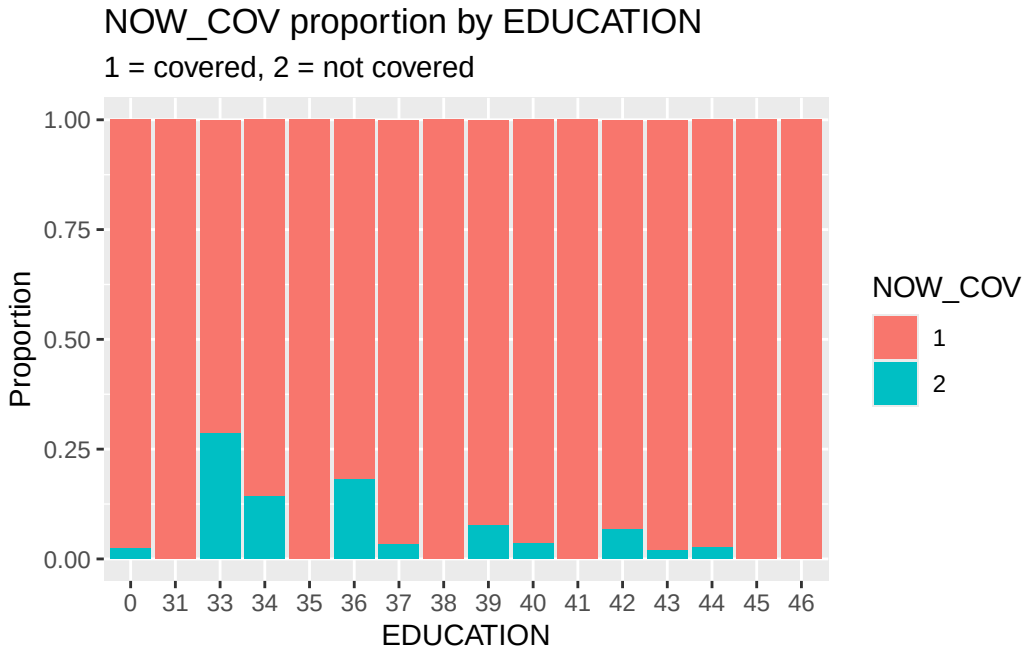
```
  <fct>   <dbl>  <dbl> <dbl> <dbl>
1 1           0     37  39.4    85
2 2           3     32  33.1    62
```




```
--- EDUCATION ---

         1     2
  0  0.975 0.025
  31 1.000 0.000
  33 0.714 0.286
  34 0.857 0.143
  35 1.000 0.000
  36 0.818 0.182
  37 0.967 0.033
  38 1.000 0.000
  39 0.924 0.076
  40 0.964 0.036
  41 1.000 0.000
  42 0.933 0.067
  43 0.980 0.020
  44 0.975 0.025
```

```
45 1.000 0.000
46 1.000 0.000
```



Our group will be using demographic and income features from the U.S. Census Bureau data to predict whether an individual has health insurance coverage. This is a classification problem as we are trying to create a model to predict whether a person either has or does not have health insurance coverage. We have a mix of quantitative and qualitative variables, with notable variables such as income, age, and number of persons in the household under age 18 as clear quantitative variables. We also have data on sex, labor force status, the longest job class held by a worker, educational attainment, and state, all of which are important qualitative features.

Model 1.) We will first fit a simple KNN model with cross validation using just our quantitative features to predict health insurance coverage status. This will be a simple starting point from which we can develop more complex models that incorporate our qualitative features.

Model 2.) We will then fit a standard logistic regression model using both our quantitative and qualitative features to predict whether someone has healthcare coverage, using stepwise and best subset selection to find the best models in both cases. We will also leverage cross validation to iteratively split the data into testing and training sets in order to best train our model and evaluate its performance.

Model 3.) We will then leverage bootstrap sampling along with our best logistic regression model to use sampling with replacement as a strategy to estimate our logistic regression model. This will allow us to determine the accuracy and variability of our estimates by constructing

confidence intervals for our model estimates. By repeatedly resampling with replacement from the dataset, we are able to construct a sampling distribution that allows us to determine model accuracy and variability.

Model 4.) We will finally leverage tree methods to predict whether someone has healthcare coverage. With tree methods we will also be able to use the full breadth of both our qualitative and quantitative features to create a robust model to predict healthcare coverage status.